

Laboratorio 1 - Tecnologías de Microprocesamiento

1st Gabriel Ramos
Ingeniería Mecatrónica
UTEC

Fray Bentos, Uruguay
gabriel.ramos@estudiantes.utec.edu.uy

2nd Valentín Grattone
Ingeniería Mecatrónica
UTEC

Fray Bentos, Uruguay
valentin.grattone@estudiantes.utec.edu.uy

Abstract—En el siguiente documento se explica el uso y implementación del lenguaje ensamblador (ASM) en el microcontrolador ATMEGA328P para resolver las problemáticas planteadas. Se presentan las implementaciones de una lavadora automática y sus diferentes modos de funcionamiento, un contador digital con 3 botones y un display de 7 segmentos, un sistema de selección y ejecución secuencia de leds y una Unidad Aritmético lógica (ALU).

Index Terms—Lavadora automática, Contador, Secuencia de leds, ALU, Registros, display.

I. INTRODUCCIÓN

En la ingeniería mecatrónica, entender cómo se comunican el software y el hardware a bajo nivel es fundamental para diseñar sistemas eficientes. Aunque hoy en día es común programar en lenguajes de alto nivel como C o C++, el uso de lenguaje ensamblador permite comprender la arquitectura interna de un microcontrolador, optimizar el uso de sus recursos y tener un control total sobre las señales del sistema. Este laboratorio se centra en el trabajo con el microcontrolador ATmega328P. A lo largo de las actividades, se realiza la configuración y manipulación de los puertos de entrada/salida digital, el uso de los registros de propósito general, la lectura del registro de estado (SREG) y la implementación de retardos por software. Para poner a prueba estos conceptos, se desarrollaron diferentes problemas prácticos en ensamblador: la automatización de una lavadora, un contador digital en un display de 7 segmentos, el control de secuencias luminosas con LEDs y el diseño de una Unidad Aritmético-Lógica (ALU) de 4 bits. En las siguientes secciones se detallan el diseño de los algoritmos, los diagramas de estado, la estructura del código y los resultados obtenidos tanto en simulación como en el montaje físico.

II. OBJETIVOS

A. Objetivo General

Programar el microcontrolador ATmega328P utilizando lenguaje ensamblador para manipular los puertos de entrada/salida.

B. Objetivos Específicos

- Configurar los registros de los puertos E/S (DDR_x, PORT_x, PIN_x) para la lectura correcta de botones y el control de salidas como LEDs y displays de 7 segmentos.

- Diseñar e implementar una máquina de estados en ensamblador que permita controlar los ciclos y secuencias de un proceso automatizado.
- Programar operaciones aritméticas y lógicas en el ATmega328P verificando la respuesta de las banderas del registro de estado SREG (*C*, *N*, *Z*).

III. MATERIALES

Microcontrolador ATmega328P.

Protoboard

Fuente de alimentación

Pulsadores

LEDs

Display 7 segmentos

Resistencias

Cables

IV. MARCO TEÓRICO

A. Lenguaje Ensamblador

En el mundo de los lenguajes de programación hay lenguajes de bajo y alto nivel, dentro de ellos está el lenguaje Ensamblador o Assembler, considerado un intermediario entre el Código de máquina y los lenguajes de alto nivel. En otras palabras, es la capa de programación que tiene el acceso más cercano al hardware, este lenguaje usa mnemónicos u opcodes que luego son traducidos a lenguaje máquina. Con ensamblador obtenemos un control detallado de hardware, optimizando los recursos en general. Una característica de este lenguaje es que cada microprocesador (uP) o microcontrolador (uC) tienen su propio lenguaje ensamblador. Al ser un lenguaje no estandarizado, este cambiará entre uP o uC, pero en general se pueden agrupar en varias categorías comunes, como lo pueden ser Operaciones con enteros (Uso de la ALU), Operaciones de mover datos (registros y memoria, pila (stack)), Operaciones I/O, Control de flujo de programa, Operaciones con números reales (aritméticas, trigonométricas, Log, Pot, raíz...), etc. Para su correcto funcionamiento, los microcontroladores requieren la configuración adecuada de los registros. Estos registros están especificados en el datasheet[1]. Uno de ellos es el SREG, un registro de 8 bits donde cada bit actúa como un indicador o flag que se modifica según el resultado de instrucciones previas.

B. Microcontrolador ATMEGA328P

Un microcontrolador es un circuito integrado que puede ser programable y que ejecuta instrucciones u órdenes que están almacenada en su memoria. Entre sus características, se encuentran el bajo consumo, algunos de ellos están en el orden de microvatios (uW) y en estado de reposo pueden llegar a consumir nanovatios (nW) lo que los hace ideales para dispositivos de bajo consumo. Existen dos grandes arquitecturas en los microcontroladores y en los microprocesadores las cuales son de Von Neuman y Harvard, se diferencian en que una utiliza la memoria en su totalidad para almacenar datos e instrucciones y la otra divide la memoria en dos, una mitad para datos y otra para instrucciones. Los microcontroladores se compone de elementos variados y que cada fabricante adapta para sus propias necesidades, pero entre las mas comunes se encuentran los registros, Unidad de control, ALU y buses. Para este laboratorio se utilizara el ATmega328P que es un microcontrolador de 8 bits desarrollado por Atmel (ahora Microchip Technology). Es ampliamente conocido por ser el "cerebro" de la placa de desarrollo Arduino Uno. Su diseño está basado en la arquitectura AVR RISC, lo que significa que ejecuta instrucciones eficientes en un solo ciclo de reloj. Esta integrado con 32 KB de memoria Flash, 2 KB de SRAM, 1 KB de EEPROM y periféricos clave como un conversor analógico-digital (ADC) de 10 bits, canales PWM y protocolos de comunicación (UART, SPI, I2C).

V. METODOLOGÍA

Para el desarrollo de cada etapa, la explicación se dividirá en tres secciones principales: código, implementación física y resolución de inconvenientes.

A. Parte A: Lavadora automática

1) **Código:** Para el diseño del código de esta lavadora automática, primero se debe modelar el diagrama de maquinas de estados para facilitar su implementación. En este caso será una **Maquina de Moore**, es decir solo depende de su estado actual. Los estados identificados fueron: Estado inicial, llenado de agua, proceso de lavado, proceso de centrifugado, proceso de secado y fin de proceso (ver Fig. 1.). Dentro del estado inicial se requiere que se indique que la lavadora esta lista para lavar $LED_Listo = 1$. Además, se cuenta con un led que indica que carga se seleccionará $LED_Carga_Ligera/Media/Pesada$ donde la carga ligera es la carga predeterminada. Se pasará al siguiente estado una vez se presione el pulsador de inicio $Boton_inicio = 1$ y la puerta esté cerrada $Puerta_cerrada = 1$ ya que el lavado no puede comenzar a menos que la puerta se encuentre cerrada. Asimismo, comienza el llenado de agua $Válvula_Agua = ON$; una vez que el sensor de agua indique que esta lleno $Sensor_agua = lleno$ se pasa al siguiente estado. La siguiente etapa es el proceso de lavado, donde se tiene el led indicador de este estado $LED_Lavado = 1$; el motor esta girando, donde realiza un giro durante 2 segundos, hace un pausa de 1 segundo y continua este proceso repitiéndolo 5 veces. Para cada carga este tiempo aumenta

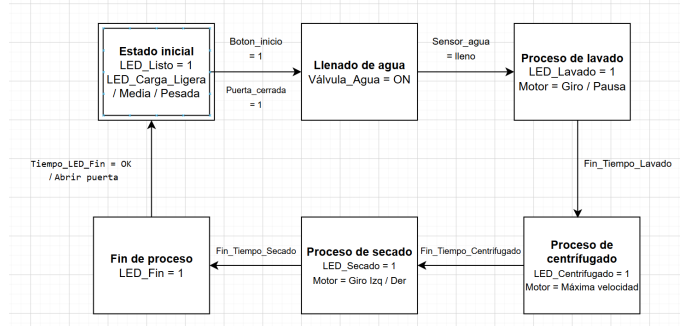


Fig. 1. Diagrama de maquinas de estados: Lavadora automática

ya que se tiene mayor peso, por lo tanto se demora mas el proceso. Para carga media son 3 segundos de giro, 2 de pausa; para carga pesada son 4 segundos de giro, 3 de pausa. Una vez termina este proceso Fin_Tiempo_Lavado , continua el proceso centrifugado, donde el motor estará a su máxima velocidad durante 15 segundos (se ajusta por carga, para cada carga aumenta 3 segundos; carga media 18 segundos; carga pesada 21 segundos) y este se representa mediante el un led $LED_Centrifugado = 1$. Cuando se culmina el proceso de centrifugado $Fin_Tiempo_Centrifugado$ inicia el proceso de secado, donde el motor gira tanto a derecha como izquierda (5 segundos por lado con pausa de 3 segundos y un ajuste de 2 segundos de aumento por carga; carga media 7 segundos por lado, 5 de pausa; carga pesada 9 segundos por lado, 7 de pausa) para poder secar la ropa, también indicado con un led $LED_Secado = 1$. Por último, luego de finalizar el secado Fin_Tiempo_Secado se enciende un led que indica que finalizo el proceso de lavado $LED_Fin = 1$.

La dificultad de la implementación en ensamblador de esta maquina de estados está en los tiempos tanto de giros y pausas, para esto se deben utilizar los **retardos** o **delays**. Asimismo, se debe asignar los puertos tanto de entrada como de salida para los pulsadores y LEDs; usando los puertos B como salidas hacia los LEDs de proceso (pines 8 a 12), los puertos D como entradas de (pines 2 al 5) y los puertos C como salida a los LEDs de selección de carga (pines A0 a A2). Además, se debe tener en cuenta el rebote que se genera al presionar el pulsador, por tanto, se debe implementar un efecto antirrebote para filtrar cualquier ruido que afecté nuestro circuito, para esto también se requiere un delay de poca duración. A continuación, se presentan los calculos para determinar los delays necesarios: Para una $f_{clk} = 16 \text{ MHz}$, el periodo de un ciclo de reloj es:

$$T_{ciclo} = \frac{1}{16 \text{ MHz}} = 0.0625 \mu s \quad (1)$$

Utiliza un doble bucle de decremetación.

- **Bucle Interno** ($r21 = 250$): Ejecuta las instrucciones `dec` (1 ciclo) y `brne` (2 ciclos si salta, 1 si termina).

$$C_{int} = (250 \times 3) - 1 = 749 \text{ ciclos} \quad (2)$$

- **Bucle Externo** ($r20 = 150$):

$$C_{ext} = 150 \times (749 + 1 + 3) - 1 = 112949 \text{ ciclos} \quad (3)$$

Sumando la llamada (`rCALL`, 3 ciclos) y retorno (`ret`, 4 ciclos), se obtiene un total de 112 957 ciclos, equivalente a:

$$t_{CORTO} = 112\,957 \times 0.0625 \mu s \approx 7.06 \text{ ms} \quad (4)$$

Consiste en un triple bucle anidado ($r20 = 82$, $r21 = 43$, $r22 = 0 \rightarrow 256$):

$$C_{1S} \approx 82 \times [43 \times (256 \times 3) + 3] \approx 16\,000\,000 \text{ ciclos} \quad (5)$$

$$t_{1S} = 16\,000\,000 \times 0.0625 \mu s = 1.00 \text{ s} \quad (6)$$

La Tabla I resume las rutinas de tiempo calculadas.

TABLE I
RESUMEN DE TIEMPOS DE EJECUCIÓN

Rutina (Registros)	Ciclos Total	Tiempo Total
DELAY_CORTO ($r20 = 150, r21 = 250$)	112 957	7.06 ms
DELAY_1S ($r20 = 82, r21 = 43, r22 = 0$)	16×10^6	1.00 s
ESPERAR_SEGUNDOS (según $r19$)	$r19 \times 16 \times 10^6$	$r19 \times 1.00 \text{ s}$

Explicación Técnica del Código Ensamblador

Se organiza en tres bloques principales:

Configuración e Inicialización

Se configura el *Stack Pointer* y se definen las E/S:

- **Puerto B (PB0–PB4):** Salidas para el control de LEDs de proceso (Listo, Lavado, Centrifugado, Secado, Fin).
- **Puerto C (PC0–PC2):** Salidas que señalan la carga seleccionada (Ligera, Media, Pesada).
- **Puerto D (PD2–PD5):** Entradas con resistencias *Pull-Up* activadas para los pulsadores de inicio/carga y sensores de seguridad (puerta y agua).

Control de Estados y Selección de Carga

En el estado REPOSO, el programa evalúa el registro $r17$ (que almacena el tipo de carga: 0, 1 o 2) y activa la salida correspondiente en el Puerto C.

Al presionar el botón de carga (PD3), la rutina CAMBIAR_CARGA incrementa $r17$ en un módulo 3 ($0 \rightarrow 1 \rightarrow 2 \rightarrow 0$) empleando DELAY_CORTO para filtrar el rebote mecánico. Antes de iniciar el ciclo mediante el botón de inicio (PD2), la rutina VALIDAR_SENORES verifica que la puerta y el agua estén activados (GND en PD4 y PD5).

Ejecución del Ciclo y Modulación Temporal Las etapas de operación modifican las salidas del Puerto B y cargan en el registro $r19$ la duración requerida en segundos:

- **Lavado (PROCESO_LAVADO):** Ejecuta 5 ciclos alternando entre giro a la derecha y pausas.
- **Centrifugado (PROCESO_CENTRIFUGADO):** Aplica duraciones fijas asociadas a $r17$ (15, 18 o 21 segundos).
- **Secado (PROCESO_SECADO):** Alterna giro derecha, pausa y giro izquierda.

La modulación del tiempo se logra ajustando $r19$ dinámicamente según la carga seleccionada:

$$t_{espera} = r19 \times 1.00 \text{ s} \quad (7)$$

Implementación de Retardos Los retardos se generan mediante bucles de espera activa:

- **DELAY_CORTO ($\approx 7.06 \text{ ms}$):** Doble bucle anidado ($r20 = 150$, $r21 = 250$) diseñado para la eliminación de rebotes en botones.
- **DELAY_1S (1.00 s):** Triple bucle anidado ($r20 = 82$, $r21 = 43$, $r22 = 0 \rightarrow 256$) que suma 16×10^6 ciclos de reloj.
- **ESPERAR_SEGUNDOS:** Llama recursivamente a DELAY_1S decrementando $r19$ hasta llegar a cero, logrando delays escalables en segundos.

2) *Implementación física:* Antes de implementar el código físicamente se debe simular, para esto se utilizó el software **Picsimlab**, donde el circuito funcionó adecuadamente, lo que dio paso a la implementación física. En la implementación física de esta lavadora automática se debe utilizar 8 LEDs; 5 para el proceso de lavado y 3 para selección de carga, estos últimos para una mejor visualización deben ser de un color distinto entre sí mismos y de los LEDs del proceso. Además, se requiere el uso de pulsadores para dar inicio al proceso, seleccionar la carga, confirmar que los sensores indiquen que la puerta esté cerrada y que la lavadora tenga agua. En el caso de los sensores, es más conveniente utilizar interruptores, pero en este caso no se contaba con ellos, por consiguiente se utilizaron pulsadores. Asimismo, para la protección de las salidas digitales de los Puertos B y C frente a un exceso de corriente, se utilizaron resistencias limitadoras de 220Ω . Aplicando la Ley de Ohm para un voltaje de alimentación $V_{CC} = 5 \text{ V}$ y una caída de tensión típica en el LED de $V_D \approx 2.0 \text{ V}$:

$$I_{LED} = \frac{V_{CC} - V_D}{R} = \frac{5 \text{ V} - 2.0 \text{ V}}{220 \Omega} \approx 13.64 \text{ mA} \quad (8)$$

La corriente obtenida (13.64 mA) asegura un brillo visual óptimo en los indicadores del proceso mientras permanece por debajo del límite máximo de corriente continua por pin (20 mA) especificado para el ATmega328P.

El circuito se implementa utilizando la placa Arduino UNO con el código previamente cargado. Luego, se debe montar el circuito con los LEDs y pulsadores sobre un protoboard; a través de resistencias y el uso de cables jumpers macho-macho se conectan las E/S anteriormente detalladas. Por último, este circuito se alimenta del mismo Arduino, usando los pines de 5V y GND.

3) *Resolución de inconvenientes:* Los inconvenientes principales fueron el largo del código y la implementación de los delays. Originalmente el código fue más largo que el que se presentó finalmente, para reducir las líneas de código, se reconstruyó mediante el uso del Stack Pointer (pila), ya que esto permitía usar las instrucciones `rCALL` y `ret` lo que facilita el uso de delays a su vez que reduce las líneas de código a utilizar.

Otros inconvenientes dentro del código fueron errores de sintaxis que mantuvieron el código en pausa por no encontrar el error, pero fue más que nada escribir nombres de forma incorrecta.

B. Parte B: Contador digital con tres botones y display de 7 segmentos

1) **Código:** Como se ha hecho anteriormente, para esta instancia se realiza la maquina de estados correspondiente. Para este caso tenemos un sistema que recibe como entrada tres botones, encargados del incremento, decremento y reset, esto se visualiza en un display de 7 segmentos. En este caso los estados son $E_0...E_9$ donde para pasar al siguiente estado se requiere pulsar incremento $\text{Boton_Inc} = 1$, para volver al estado anterior se requiere pulsar decremento $\text{Boton_Dec} = 1$ y para volverá al valor inicial (E_0) se debe pulsar reset $\text{Reset} = 1$. En el caso de que no se pulse ningún botón permanecerá en el mismo estado a la espera de una entrada Esperar_Boton .

Puede surgir la duda de que pasaría al pulsar incremento estando en el valor mas alto E_9 o pulsar decremento en el valor más bajo E_0 . En este caso permanecerá a la espera, ya que a nivel de código se establecen como limites inferior y superior los valores de 0 y 9 respectivamente.

Control de Límites para Incremento y Decremento

Puede surgir la duda de que pasaría al pulsar incremento estando en el valor mas alto E_9 o pulsar decremento en el valor más bajo E_0 . En este caso permanecerá a la espera, ya que a nivel de código se establecen como limites inferior y superior los valores de 0 y 9 respectivamente ($0 \leq r17 \leq 9$). Esto se implementó mediante la instrucción de comparación inmediata CPI (*Compare with Immediate*) y saltos condicionales (ver Listing 1).

Al presionar el botón de incremento, el microcontrolador verifica si el registro ha alcanzado el valor máximo permitido (9). Si la condición se cumple, el salto BREQ redirige la ejecución al final de la rutina evitando la instrucción INC . De manera análoga, durante el decremento se evalúa si el valor actual es 0 antes de ejecutar la instrucción DEC .

Listing 1. Lógica de control de límites superior e inferior del contador

```
1 ;==== CONTROL DE L MITE SUPERIOR (
  INCREMENTO) ====
2 cpi r17, 9 ; Compara si el
  contador llego a 9
3 breq fin_inc ; Si r17 == 9, salta
  sin incrementar
4 inc r17 ; Incrementa el
  contador (r17 = r17 + 1)
5 fin_inc:
6 rjmp bucle_principal
7
8 ;==== CONTROL DE L MITE INFERIOR (
  DECREMENTO) ====
9 cpi r17, 0 ; Compara si el
  contador esta en 0
10 breq fin_dec ; Si r17 == 0, salta
  sin decrementar
11 dec r17 ; Decrementa el
  contador (r17 = r17 - 1)
12 fin_dec:
13 rjmp bucle_principal
```

En la implementación del código se utilizaron los puertos D como salidas para maneja el display y el puerto C se configuro

como entrada donde también se activan las resistencia pull ups para que los botones lean 1 lógico en estado de reposo y 0 al presionarse. Además, se utilizo el $r17$ inicializado en 0 para almacenar el estado actual del contador.

Para garantizar una lectura correcta de los pulsadores y quitar el efecto de rebote mecánico, se implementó una rutina de delay mediante basada en tres bucles anidados utilizando los registros $r18$, $r19$ y $r20$.

Para $f_{clk} = 16 \text{ MHz}$ ($T_{clk} = 62.5 \text{ ns}$).

1. Bucle Interno (r20) El registro $r20$ se inicializa en 0, por lo que al decrementar realiza un desbordamiento a 255_{10} ($0xFF$), ejecutando 255 iteraciones:

$$N_{int} = (254 \times (1_{DEC} + 2_{BRNE})) + (1 \times (1_{DEC} + 1_{BRNE})) = 764 \text{ ciclos} \quad (9)$$

2. Bucle Medio (r19) El bucle intermedio ejecuta 150 iteraciones del bucle interno, añadiendo sus propias instrucciones de control:

$$N_{med} \approx 150 \times (N_{int} + 1_{DEC} + 2_{BRNE}) = 150 \times 767 = 115,050 \text{ ciclos} \quad (10)$$

3. Bucle Externo (r18) y Tiempo Total El bucle externo repite la rutina completa 2 veces. El tiempo de retardo equivalente t_{delay} se calcula mediante:

$$N_{total} \approx 2 \times N_{med} = 230,100 \text{ ciclos} \quad (11)$$

$$t_{delay} = \frac{N_{total}}{f_{clk}} = \frac{230,100 \text{ ciclos}}{16 \times 10^6 \text{ Hz}} \approx 14.38 \text{ ms} \quad (12)$$

Puesto que el código ejecuta esta rutina dos veces por cada interacción (una al detectar la presión de la tecla y otra al confirmar su liberación), el tiempo total queda aproximadamente en 28.76 ms.

2) **Implementación física:** Este circuito se construye utilizando tres botones, resistencias y un display de 7 segmentos (se contaba con ánodo común). Para conectar cada salida con su correspondiente entrada en el display se utiliza una resistencia, en este caso se utilizo el mismo valor de resistencia que se uso en la parte A, 220Ω para cada entrada, y una resistencia desde V_{cc} . Por ultimo, se conectan los botones con sus respectivas entradas.

3) **Resolución de inconvenientes:** Uno de los inconvenientes principales fue que el código no funcionaba en el software simulación PICSIMLab, por lo tanto se diseño un código similar pero con un cambio, que fue utilizar el complemento a uno con los bits, para PICSIMLab era obligatorio invertir los bits porque el módulo del simulador utiliza una pantalla de Ánodo Común, mientras que la lógica del código original estaba escrita para Cátodo Común. Luego, esto ultimo fue otro inconveniente, al momento de implementar el código en físico, ya que como se menciono anteriormente, se contaba con un display ánodo común.

C. Parte C: Selección y ejecución de secuencia de LEDs

1) **Código:** Para el diseño de esta práctica se modeló el sistema como una **Máquina de Moore jerárquica**, en la cual

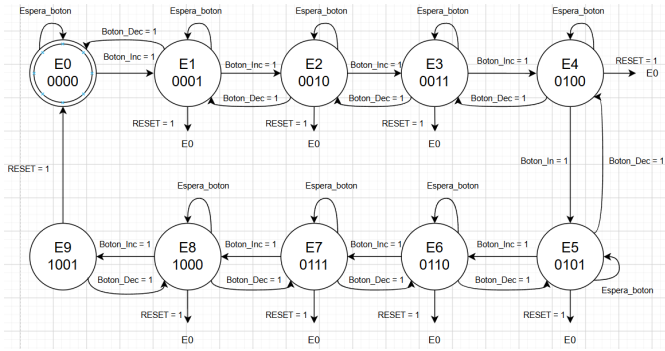


Fig. 2. Diagrama de máquinas de estados: Contador digital y display

la salida (el patrón de encendido de los 8 LEDs en el Puerto D) depende únicamente del estado actual y no de las entradas de forma instantánea; los pulsadores actúan solo como disparadores de transición entre estados. Se implementaron ocho estados principales, Secuencia_1 a Secuencia_8 (ver Fig. 3), cada uno asociado aun patrón luminoso distinto que se repite de forma cíclica mediante una micro-secuencia interna de sub-patrones temporizados con la subrutina Pausa. Al finalizar la última sub-salida de un estado, un `rjmp` retorna al inicio del mismo, generando una ejecución continua hasta que el usuario provoque una transición externa.

Las transiciones se gestionan mediante el registro de índice `r17` (inicializando en 1) y el bloque Selector, que evalúa dicho índice mediante una cascada `cpi/brne/rjmp`. El primer pulsador (`PC0`, Siguiente) incrementa `r17`, el segundo (`PC1`, Anterior) lo decrementa; y el tercero (`PC2`, Reset) fuerza `r17 = 1` desde cualquier estado (transición global, ver Fig. 3, rama "Reset → Secuencia 1"). Ambas transiciones incluyen manejo de desbordamiento circular: si `r17 > 8` se reinicia a 1, y si `r17 < 1` se fuerza a 8. Cada transición se ejecuta únicamente tras detectar la liberación del pulsador, evitando incrementos múltiples por una única pulsación sostenida.

A continuación se presentan los cálculos para determinar el retardo utilizado en cada patrón. Para $f_{clk} = 16$ MHz, el periodo de ciclo es $T_{ciclo} = 0.0625 \mu s$ (conforme a (1)). A diferencia del doble bucle empleado en la Parte A, la subrutina Pausa utiliza un **triple bucle anidado** ($r18 = 50$, $r19 = 100$, $r20 = 200$) para alcanzar un retardo del orden de los 188 ms sin desbordar registros de 8 bits:

- **Bucle Interno** ($r20 = 200$): Ejecuta `dec` (1 ciclo) y `brne` (2 ciclos si salta, 1 si termina).

$$C_{int} = (200 \times 3) - 1 = 599 \text{ ciclos} \quad (13)$$

- **Bucle Medio** ($r19 = 100$):

$$C_{med} = 100 \times (599 + 1 + 1 + 2) - 1 = 60\,299 \text{ ciclos} \quad (14)$$

- **Bucle Externo** ($r18 = 50$): Incluye la lectura intercalada de los tres pulsadores (`sbic`, 1 ciclo cada uno en reposo).

$$C_{ext} = 50 \times (60\,299 + 1 + 3 + 1 + 2) - 1 = 3\,015\,299 \text{ ciclos} \quad (15)$$

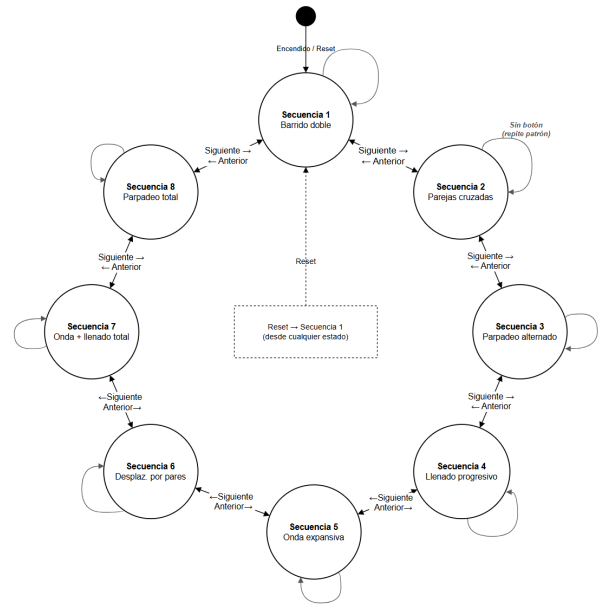


Fig. 3. Diagrama de máquina de estados: Secuencia de LEDs

Sumando la carga inicial (`ldi`, 1 ciclo) y el retorno (`ret`, 4 ciclos), se obtiene un total de 3 015 304 ciclos, equivalente a:

$$t_{PAUSA} = 3\,015\,304 \times 0.0625 \mu s \approx 188.46 \text{ ms} \quad (16)$$

La Tabla III resume el número de llamadas a Pausa y el tiempo total de ciclo completo para cada secuencia.

TABLE II
RESUMEN DE TIEMPOS POR SECUENCIA

Secuencia	Llamadas a Pausa	Tiempo Total
1 – Barrido doble	7	1.32 s
2 – Parejas cruzadas	7	1.32 s
3 – Parpadeo alternado	2	0.38 s
4 – Llenado progresivo	16	3.02 s
5 – Onda expansiva	18	3.39 s
6 – Desplaz. por pares	17	3.20 s
7 – Onda + llenado total	14	2.64 s
8 – Parpadeo total	6	1.13 s

Adicionalmente, dado que la lectura de pulsadores ocurre al finalizar cada iteración del bucle externo, el sistema muestrea los botones cada $C_{med} \times T_{ciclo} \approx 3.77$ ms, brindando una respuesta prácticamente inmediata al usuario pese a que el patrón visual solo cambia cada ~ 188 ms.

Explicación Técnica del Código Ensamblador

Se organiza en tres bloques principales:

Configuración e Inicialización

Se configura el *Stack Pointer* y se definen las E/S:

- **Puerto D (PD0–PD7):** Salida hacia los 8 LEDs, inicializado en `0x00`.
- **Puerto C (PC0–PC2):** Entradas para los pulsadores Siguiente, Anterior y Reset. A diferencia de la Parte A, aquí se deshabilitan las resistencias de *Pull-Up* internas

(PORTC=0x00), por lo que la detección se realiza en lógica activa en alto, requiriendo resistencias de *pull-down* externas.

- Se inicializa el registro de estado $r17 \leftarrow 1$, seleccionando *Secuencia_1* por defecto.

Bloque de Selección (Selector)

En cada reingreso a esta etiqueta se reinicializa el *Stack Pointer* (medida defensiva ante el uso de *rcall/ret*) y se evalúa $r17$ mediante la cascada *cpi/brne/rjmp*, redirigiendo la ejecución al estado luminoso correspondiente.

Bloque de Ejecución y Temporización

Cada rutina *Secuencia_X* carga sucesivos patrones de bits (*ldi* con máscaras $1 \ll \text{PDn}$ combinadas mediante $|$), los escribe en *PORTD* y llama a *Pausa*, retornando al inicio del propio estado mediante *rjmp*. La subrutina *Pausa* implementa el triple bucle descrito y, en cada vuelta del bucle externo, intercala la lectura de los tres pulsadores mediante *sbic*; al detectar una pulsación, abandona el retardo en curso saltando a las rutinas *Siguiente*, *Anterior* o *Reset*, cada una de las cuales implementa espera activa hasta la liberación del botón antes de modificar $r17$ y retornar a *Selector*.

2) *Implementación física*: Se requieren 8 LEDs (uno por bit del Puerto D) con sus respectivas resistencias limitadoras, 3 pulsadores (*Siguiente*, *Anterior*, *Reset*) conectados a PC0–PC2, y 3 resistencias de *pull-down* externas dado que el firmware no habilita las resistencias internas del microcontrolador. Aplicando la Ley de Ohm con $V_{CC} = 5 \text{ V}$ y $V_D \approx 2.0 \text{ V}$ sobre resistencias de 220Ω :

$$I_{LED} = \frac{V_{CC} - V_D}{R} = \frac{5 \text{ V} - 2.0 \text{ V}}{220 \Omega} \approx 13.64 \text{ mA} \quad (17)$$

Corriente inferior al límite de 20 mA por pin del ATmega328P, con un margen de seguridad de 6.36 mA. Cada LED se conecta en serie con su resistencia entre el pin del Puerto D y GND; cada pulsador se conecta entre V_{CC} y su pin de entrada, con la resistencia de *pull-down* entre dicho pin y GND. El circuito se alimenta desde los pines 5V y GND del Arduino Uno.

3) *Resolución de inconvenientes*: El principal inconveniente fue el rebote mecánico de los pulsadores, que Provocaba múltiples incrementos o decrementos no deseados del índice de secuencia. Se resolvió mediante una espera activa de liberación (*sbic* combinado con *rjmp*) que bloquea el avance del programa hasta que el pulsador regresa establemente a su estado de reposo, garantizando una única transición por pulsación sin importar el tiempo que el usuario mantenga presionado el botón.

Otro inconveniente fue evitar que el índice de secuencia $r17$ escapara del rango válido [1, 8] al incrementarse o decrementarse de forma independiente en cada pulsación. Se solucionó incorporando una verificación explícita mediante *cpi/brne/ldi* inmediatamente después de cada modificación de $r17$, forzando el retorno circular del índice y evitando que la cascada de comparaciones del bloque *Selector* quedara sin coincidencia.

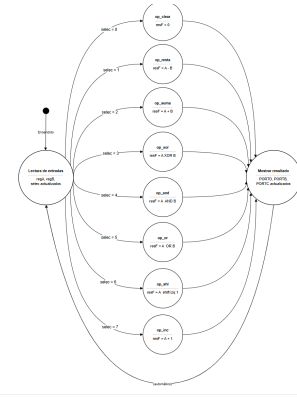


Fig. 4. Diagrama de máquina de estados: ALU

D. Parte D: Unidad Lógica Aritmética (ALU)

1) *Código*: El firmware de esta práctica se estructura como una **Máquina de Moore de tres fases**: un estado de lectura, ocho estados de operación mutuamente excluyentes y un estado de presentación de resultados (ver Fig. 4), enlazados de forma automática en un único ciclo continuo. En el estado *Lectura de entradas* se capturan los operandos de 4 bits A y B y la señal de selección S de 3 bits desde *PIND*, *PINB* y *PINC* respectivamente, enmascarando los bits no utilizados mediante *andi*. Según el valor de S , una cascada *cpi/brne* redirige la ejecución hacia el estado *op_X* correspondiente (*op_clear* a *op_inc*), el cual calcula el resultado F conforme a la Tabla III. Finalmente, el estado *Mostrar resultado* captura el registro *SREG*, formatea F , las banderas y el código de operación, y retorna automáticamente a *Lectura de entradas*.

TABLE III
TABLA DE VERDAD DE LA ALU

S2	S1	S0	Operación	Instrucción AVR
000			CLEAR	clr resF
001			$A - B$	sub resF, regB
010			$A + B$	add resF, regB
011			XOR	eor resF, regB
100			AND	and resF, regB
101			OR	or resF, regB
110			SHL	lsl resF
111			INC	inc resF

Un aspecto crítico del diseño es que las instrucciones aritméticas del ATmega328P operan sobre registros de 8 bits, mientras que la ALU trabaja únicamente sobre el nibble bajo (4 bits, rango 0-15). Para $A = 13$ (1101) y $B = 7$ (0111):

$$A + B = 13 + 7 = 20 = 00010100_2 \quad (18)$$

El resultado excede el rango de 4 bits (bit 4 = 1), pero la instrucción *add* solo activa el acarreo nativo ante un desborde del bit 7, que aquí no ocurre. Por ello, el firmware evalúa explícitamente dicho bit (*sbrc resF, 4*) y, de estar activo, fuerza la bandera de Carry mediante *sec antes* de aplicar la máscara *andi resF, 0x0F*, que conserva $F = 0100$

(4). El mismo criterio se aplica en `op_shl`. En cambio, la instrucción `sub` refleja correctamente el préstamo de 4 bits de forma nativa, por lo que `op_resta` no requiere corrección. Las operaciones lógicas (XOR, AND, OR) no modifican la bandera *C* conforme al *datasheet* AVR, comportamiento que se respeta sin forzarlo artificialmente. La Tabla IV resume el comportamiento de las banderas para el ejemplo numérico anterior.

TABLE IV
COMPORTAMIENTO DE BANDERAS ($A = 13$, $B = 7$)

Operación	Z	N	C
CLEAR	1	0	0
$A - B$	0	0	0
$A + B$	0	0	1 (forzado)
XOR	0	1	sin cambio
AND	0	0	sin cambio
OR	0	1	sin cambio
SHL	0	1	1 (forzado)
INC	0	1	sin cambio

Explicación Técnica del Código Ensamblador

Se organiza en tres bloques principales:

Configuración e Inicialización

- **Puerto D (PD0–PD3 entrada / PD4–PD7 salida):** Operando *A* con *pull-up* interno; salida del resultado *F*.
- **Puerto B (PB0–PB3 entrada / PB4–PB6 salida):** Operando *B* con *pull-up* interno; salida de las banderas *C*, *N*, *Z*.
- **Puerto C (PC0–PC2 entrada / PC3–PC5 salida):** Selector *S* con *pull-up* interno; salida del código de operación.

A diferencia de la Parte C, aquí sí se emplean las resistencias de *pull-up* internas del ATmega328P para todas las entradas.

Bloque de Control Principal (**bucle_principal**)

En cada interacción se leen los tres puertos de entrada, actualizando *regA*, *regB* y *selec*, y una cascada `cpb/breq` redirige el flujo hacia la operación seleccionada según la Tabla III. Tras `mostrar_resultado`, un `rjmp bucle_principal` reinicia el ciclo, actualizando el resultado en tiempo real ante cualquier cambio de las entradas.

Bloque de Ejecución de Operaciones y Banderas

Cada rutina `op_X` copia *A* a *resF* (`mov`) y aplica la operación correspondiente, incorporando en `op_suma` y `op_shl` la corrección manual del acarreo de 4 bits (`sbrc/sec`) antes del enmascarado `andi resF, 0x0F`. Al finalizar, `mostrar_resultado` captura SREG mediante `in reg_flags, SREG` como primera instrucción, antes de que las instrucciones de formateo (`lsl` repetidos para reubicar los bits en el nibble superior de cada puerto) puedan alterar las banderas recién calculadas, presentando finalmente el resultado, las banderas y el selector en PORTD, PORTB y PORTC.

2) *Implementación física:* Se requieren 4 dip-switches para el operando *A*, 4 para el operando *B*, 3 para el selector *S*, y LEDs indicadores para el resultado *F* (4 bits), las banderas *C/N/Z* (3 bits) y el código de operación (3 bits), con sus

respectivas resistencias de 220 Ω . Aplicando la misma Ley de Ohm que en la Parte C se obtiene $I_{LED} \approx 13.64$ mA, dentro del límite de 20 mA por pin del ATmega328P. Las entradas se conectan entre su pin correspondiente y GND, aprovechando las resistencias de *pull-up* internas habilitadas por software (sin resistencias externas adicionales). Los LEDs de salida se conectan en serie con su resistencia entre el pin de salida y GND, alimentando todo el circuito desde los pines 5V y GND del Arduino Uno.

3) *Resolución de inconvenientes:* El principal inconveniente fue conciliar el acarreo de 4 bits exigido por la ALU con el acarreo nativo de 8 bits del hardware AVR, ya que instrucciones como `add` y `lsl` solo activan la bandera *C* ante un desborde del bit 7. Se resolvió evaluando explícitamente el bit 4 del resultado intermedio mediante `sbrc resF, 4` y forzando la bandera con `sec` antes del enmascarado final, logrando que *C* refleje fielmente el comportamiento de una ALU real de 4 bits.

Otro inconveniente fue evitar que las instrucciones de formateo de salida corrompieran las banderas antes de su lectura, dado que los desplazamientos `lsl` utilizados para reubicar los bits en cada puerto modifican por sí mismos el registro SREG. Se solucionó capturando dicho registro en `reg_flags` mediante `in reg_flags, SREG` como primera instrucción del estado `Mostrar resultado`, inmediatamente después del salto proveniente de cualquiera de las rutinas de operación, asegurando la integridad de las banderas antes de cualquier manipulación posterior.

VI. CONCLUSIÓN

En el desarrollo de este laboratorio se cumplieron con éxito todos los objetivos planteados, tanto el general como los específicos. A través de las cuatro prácticas (lavadora automática, contador digital, secuencia de LEDs y ALU), se logró implementar la programación a bajo nivel en el microcontrolador ATmega328P utilizando lenguaje ensamblador. Se validó correctamente la interacción entre software y hardware, logrando configurar los puertos de entrada/salida para la lectura de botones y el control de LEDs y displays. Asimismo, se implementaron máquinas de estados para controlar procesos secuenciales y se ejecutaron operaciones aritmético-lógicas verificando la respuesta de las banderas del registro de estado (SREG). Todas las aplicaciones funcionaron de forma satisfactoria tanto en la simulación como en la implementación física, demostrando la importancia de comprender la arquitectura interna del procesador para diseñar sistemas mecatrónicos eficientes.

VII. BIBLIOGRAFÍA

- Atmel Corporation. (2015). *8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash: ATmega328P datasheet* (Doc. 7810D-AVR-01/15). ALLDataSheet. <https://www.alldatasheet.com/htmlpdf/241077/ATMEL/ATMEGA328P/312>
- Arduino LLC. (s. f.). *Arduino Uno board datasheet*. ALLDataSheet. Recuperado el 1 de septiembre

de 2026, de <https://www.alldatasheet.com/datasheet-pdf/view/1943445/ARDUINO/ARDUINO-UNO.html>
Toshiba Corporation. (s. f.). *TOS5161 display datasheet*. ALLDataSheet. Recuperado el 1 de septiembre de 2026, de <https://www.alldatasheet.es/datasheet-pdf/view/96548/ETC/TOS5161.html>

VIII. EVIDENCIAS

Link al drive: <https://drive.google.com/drive/folders/1L9qNOcSg-tc96hkSYotECaIoGvJigtM-?usp=sharing>

Link al repositorio en Github
<https://github.com/gabrielramos-netizen/Laboratorio1.0TMI>